CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
ENGENHARIA ELÉTRICA

CHRISTOPHER CÉSAR DE MELO ANDRADE

LABORATÓRIO DE SISTEMAS MICROPROCESSADOS
RELATÓRIO: PRÁTICA 6

Belo Horizonte, 30 de Outubro de 2025

Para todos os programas dessa prática foram ativados todos os resistores de pull-up internos na PORTA B, além de configurar toda a porta como entrada, como solicitado pelo fabricante do *KIT XM118*, além disso, todos os pinos da porta foram definidos como digitais. Também, foi considerado usar os pinos *RE0* e *RE1* para os LEDs Bicolor e *RC2* para o buzzer, definindo-os como saída. Para que as interrupções funcionassem, definiu-se toda a PORTA B como entrada.

1.

```c
/*
 * File:   ex01.c
 * Author: Christopher Andrade
 *
 * Created on 30 de Outubro de 2025, 10:55
 */

#include <xc.h>

#define _XTAL_FREQ 20000000

#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config MCLRE = ON
#pragma config PWRT = ON
#pragma config XINST = OFF

signed char aux = 0;

void interrupt blink(void)
{
    if(INTCONbits.TMR0IF)
    {
       INTCONbits.TMR0IF = 0;

       aux = LATEbits.LATE0;
       LATEbits.LATE0 = LATEbits.LATE1;
       LATEbits.LATE1 = aux;
       TMR0   = 26474;
    }

    if(INTCONbits.INT0IF)
    {
        __delay_ms(100);
        INTCONbits.INT0IF = 0;

        if(!T0CONbits.TMR0ON)
        {
            LATEbits.LATE0 = 1;
            LATEbits.LATE1 = 0;
            TMR0 = 26474;
            T0CONbits.TMR0ON = 1;
        }
        else
        {
            LATEbits.LATE0 = 0;
            LATEbits.LATE1 = 0;
            TMR0 = 0;
            T0CONbits.TMR0ON = 0;
        }
    }
}
```

```c
void main(void)
{
   INTCON2bits.RBPU = 0;
   ADCON1 = 0x0F;
   TRISB = 0xFF;
   TRISEbits.RE0 = 0;
   TRISEbits.RE1 = 0;
   LATEbits.LATE0 = 1;
   LATEbits.LATE1 = 0;
   INTCON = 0b11110000;
   T0CON  = 0x86;
   TMR0   = 26474;

   while(1){}
}
```

Neste programa, utiliza-se o *Timer0* configurado para gerar interrupções periódicas, responsáveis por alternar o estado dos pinos *RE0* e *RE1*, que controlam o LED bicolor.

Inicialmente, o registrador *INTCON* foi definido para ativar as interrupções globais, periféricas, do *Timer0* e *INT0*. *T0CON*, definido com 0x86, configura um *Timer0* com prescale de 1:128 com resolução de 16.

Para gerar uma interrupção por segundo encontrou-se por cálculos o valor inicial de TMR0:

$$F_{tmrclk} = \frac{\frac{20 * 10^6}{4}}{128} = 39.063$$

$$TMR0 = 65536 - \frac{39.063}{1\,Hz} = 26474$$

A cada overflow do *Timer0*, o código troca(*swap*) os estados dos LEDs, produzindo o efeito de piscar em verde e vermelho, com uma frequência próxima de 1 Hz.

A interrupção externa *INT0 (RB0)* funciona como uma chave liga/desliga do pisca LED: ao ser acionada, o programa realiza um pequeno debounce por software e então pausa ou reinicia o funcionamento do Timer0, garantindo que o LED permaneça piscando ou apagado de maneira estável conforme o estado selecionado. Dessa forma, o sistema demonstra o uso integrado de interrupção por timer e interrupção externa para controle direto de um LED bicolor.

2.

```c
/*
 * File:   ex02.c
 * Author: Christopher Andrade
 *
 * Created on 30 de Outubro de 2025, 10:55
 */

#include <xc.h>

#define _XTAL_FREQ 20000000

#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config MCLRE = ON
#pragma config PWRT = ON
#pragma config XINST = OFF

signed char aux = 0;

void interrupt blink(void)
{
    if(PIR1bits.TMR1IF)
    {
       PIR1bits.TMR1IF = 0;
       LATCbits.LATC2 = ~LATCbits.LATC2;
       TMR1   = 63263;
    }

    if(INTCONbits.INT0IF)
    {
        __delay_ms(100);
        INTCONbits.INT0IF = 0;

        if(!T1CONbits.TMR1ON)
        {
            TMR1 = 63263;
            T1CONbits.TMR1ON = 1;
        }
        else
        {
            LATCbits.LATC2 = 0;
            T1CONbits.TMR1ON = 0;
            TMR1 = 0;
        }
    }
}
void main(void)
{
   INTCON2bits.RBPU = 0;   // habilita os resistores de pull-up internos
   ADCON1 = 0x0F;          // desliga entradas analógicas
   TRISB = 0xFF;
   TRISCbits.RC2 = 0;

   INTCON = 0b11010000;
   PIE1bits.TMR1IE  = 1;
   T1CON  =  0b10000101;
   TMR1   = 63263;
   while(1){}
}
```

Neste programa, o *Timer1* é utilizado para gerar interrupções periódicas que acionam o buzzer conectado ao pino *RC2*. A cada overflow do *Timer1*, o programa inverte o nível lógico do pino, produzindo uma onda quadrada que gera o som desejado.

O valor de recarga do registrador *TMR1* é ajustado para produzir uma frequência de aproximadamente 2,2 kHz.

$$F_{tmrclk} = \frac{\frac{20 * 10^6}{4}}{1} = 5000000$$

$$TMR0 = 65536 - \frac{5000000}{2200\,Hz} = 63263$$

Assim como no exercício anterior, é utilizada a interrupção externa *INT0 (RB0)* para ligar e desligar o funcionamento do buzzer. Após um pequeno atraso para evitar bouncing, a interrupção verifica se o *Timer1* está ativo: caso não esteja, reinicia a contagem e habilita o som; caso esteja, o buzzer é silenciado e o timer interrompido. Dessa forma, o programa demonstra o uso do *Timer1* como gerador de sinal de áudio, controlado dinamicamente por uma interrupção externa.

3.

```c
/*
 * File:   ex03.c
 * Author: Christopher Andrade
 *
 * Created on 30 de Outubro de 2025, 10:55
 */

#include <xc.h>

#define _XTAL_FREQ 20000000

#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config MCLRE = ON
#pragma config PWRT = ON
#pragma config XINST = OFF

signed char aux = 0;



void __interrupt(high_priority) High(void)
```

```
{
    if (INTCONbits.TMR0IF)
    {
        INTCONbits.TMR0IF = 0;

        aux = LATEbits.LATE0;
        LATEbits.LATE0 = LATEbits.LATE1;
        LATEbits.LATE1 = aux;

        TMR0 = 26474;
    }

    if (INTCONbits.INT0IF)
    {
        __delay_ms(100);
        INTCONbits.INT0IF = 0;

        if (!T0CONbits.TMR0ON)
        {
            LATEbits.LATE0 = 1;
            LATEbits.LATE1 = 0;
            TMR0 = 26474;
            T0CONbits.TMR0ON = 1;
        }
        else
        {
            LATEbits.LATE0 = 0;
            LATEbits.LATE1 = 0;
            TMR0 = 0;
            T0CONbits.TMR0ON = 0;
        }
    }
}

void __interrupt(low_priority) Low(void)
{
    if (PIR1bits.TMR1IF)
    {
        PIR1bits.TMR1IF = 0;
        LATCbits.LATC2 = ~LATCbits.LATC2;
        TMR1 = 63263;
    }

    if (INTCON3bits.INT1IF)
    {
        __delay_ms(100);
        INTCON3bits.INT1IF = 0;

        if (!T1CONbits.TMR1ON)
        {
            TMR1 = 63263;
            T1CONbits.TMR1ON = 1;
        }
        else
        {
            LATCbits.LATC2 = 0;
            T1CONbits.TMR1ON = 0;
            TMR1 = 0;
        }
    }
}

void main(void)
{
   INTCON2bits.RBPU = 0;
```

```
   ADCON1 = 0x0F;
   TRISB = 0xFF;
   TRISEbits.RE0 = 0;
   TRISEbits.RE1 = 0;
   LATEbits.LATE0 = 1;
   LATEbits.LATE1 = 0;
   TRISCbits.RC2 = 0;

   RCONbits.IPEN = 1;

   INTCON2bits.TMR0IP = 1;

   IPR1bits.TMR1IP = 0;
   INTCON3bits.INT1IP = 0;

   INTCON = 0b11110000;
   INTCON3bits.INT1IE = 1;
   PIE1bits.TMR1IE  = 1;
   T0CON  = 0x86;
   T1CON  =  0b10000101;
   TMR0   = 26474;
   TMR1   = 63263;

   while(1){}
}
```

Para esse programa: as rotinas responsáveis pelos LEDs foram configuradas como alta prioridade, enquanto o controle do buzzer opera como baixa prioridade. Para isso, ativou-se o sistema de prioridades através do bit *IPEN* em *RCONbits.IPEN = 1*, permitindo que o PIC18 diferencie interrupções *high/low*.

Em seguida, definiu-se o *Timer0* como alta prioridade por meio do bit I*NTCON2bits.TMR0IP = 1*, a interrupção externa INT0 é definida nativamente como alta prioridade em relação as demais interrupções externas.

Por sua vezes o Timer1 e a interrupção INT1 foram configurados como baixa prioridade ajustando *IPR1bits.TMR1IP = 0* e *INTCON3bits.INT1IP = 0*.

O programa mantém a funcionalidade observada em ex01.c e ex02.c. Assim, o código demonstra o uso combinado de múltiplos timers, interrupções externas e hierarquia de prioridades do PIC18 para garantir que o piscar dos LEDs (função principal) nunca seja interrompido pelo processamento secundário do buzzer, haja vista que as funções de alta e baixa prioridade, *High*() e *Low*(), foram definidas separadamente.